

Lec 02 Activation Functions & Loss Functions in Deep Learning

Lec 02 Activation Functions & Loss Functions in Deep Learning

Lecture	02 / 63 + intro
Duration	36:34
Captions	English (auto-generated)
Words	5,942
Video	jXlaQvjQiIU
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome to the lecture series on fundamentals of generative AI and the large language models. We are in the week one. In the previous session, we had uh started with the fundamentals of generative AI. We had seen all the basic concepts. In this particular session, we will look into the activation functions and the loss functions that are needed mainly for deep learning. So, coming into the activation function, if you look into this particular diagram, what of the single circle that you are seeing, it is a neuron. In this particular neuron, there are two mathematical operations that are going to take place. The first mathematical operation is, can you able to see here? We can perform the

[00:45] multiplication of input, you're multiplying it with the weight, and then we are adding the bias. So, it is the weighted sum of input with weight. So, once we complete this particular operation, the next is we are going to pass this to an activation function. So, there has to be some role of an activation function here, here, right? Now, let us assume what if you don't have this activation function. So, before we actually get

into the various types, first we need to understand what is the need for an activation function.

[01:19] Suppose if there is no activation function which is here, what will be the output? The output will be whatever is this summation part, the same summation part will be coming as an output here. In that case, see, look at this first point. Without an activation function, the weighted sum of an input is directly passed as the output. Now, when you directly pass, what exactly is going to happen? Here, the neuron acts as a linear model.

[01:46] Therefore, the role of an activation function here is, in case if you want the non-linearity, then we need to have a non-linear activation of function. So, without the role without the activation function here the neuron is acting as an linear model. Second, this is one such neuron. Suppose, if you have five different neurons. Say for example, you have first neuron, second neuron, third, fourth, and fifth. Here also you will have an weighted sum, weighted sum. In all of these we will have the weighted sum.

[02:23] Now, whatever is this weighted sum when you pass it to an activation function, we get some output. Here we get some output. Similar way from each of these five neurons, we are going to get five different output. Now, what are we going to do in the next layer? Suppose in the next layer again if you have an you know hidden layer. Suppose again we have a few neurons in the next layer. How are we going to do as per the neural network? One neuron is connected to all other neurons.

[02:59] This output is connected to all other neurons. Similar way second neuron output is connected to all other. So, from this what are we understanding? Is it not the output of various neurons are combined together? Which means is it not it is helping us to learn the complex patterns? So, the second advantage of going or second need for an activation function is it will help us in learning the complex pattern. This is the second advantage. And the third, we have an activation functions such as the ReLU and the family of ReLU activations. That is the variants of ReLU. Now, what exactly ReLU does? It is a max of 0, X. So, if I have a positive value, it is going to send a positive value. If you have a negative value, max

[03:54] of zero and negative is a zero, which means is it not suppressing some unwanted information? So, the third advantage of going for an activation function is especially if we are using an activation functions such as the ReLU or the variants of ReLU, it is going to forward only positive information and all negative information it is going to suppress. And since the course is all about the generative AI, the role of activation functions in generative AI is it is going to help us in a transforming the noise to a meaningful representation or it is going to transform the latent to an meaningful output. So, with this what we are trying to understand is only what is the importance, what is the role

[04:40] of an activation function in neural network. The next immediate question is in the entire design of an architecture, where is that this activation functions has to be placed? Now, if you take this particular example, you can see there are hidden layers and there is an output layer also, which means the activation functions are something which can be placed which has to be placed at hidden layer as well as at the output layer.

[05:09] So, the activation functions are used at hidden layer and at the output layer. So, what are we trying to understand? Only the need and where all the activation functions can be placed. Next, if that is the case, what are the different activation functions that are there at a hidden layer and what are different activation functions are there at the output layer? Can we use the same activation functions? Answer is no. Let us try to see why there are different activation functions. First, at a hidden layer, we have a ReLU which is most popularly used to activation function.

[05:45] So, what is the formula for a ReLU activation? It is $\max(0, x)$. Now, what exactly this will do? Now, this x here is the summation part. In the neuron, we will have the summation and we are going to get some value from this summation, right? Because it is just the multiplication of input with weight and you are adding the bias. So, we are going to get a numerical value. Now, that numerical value is what we are talking about the x . Now, this x now when you pass it to a ReLU, which is having a formula $\max(0, x)$, let us try to understand how ReLU works. Suppose if the input is a negative, whatever we are seeing here are the different examples.

[06:28] If the input is a negative, when we write a maximum of 0 with a negative value, so definitely the maximum between 0 and a negative value is 0, right? So, we have got 0. Now, what if summation itself is a negative value? Then, $\max(0, x)$, 0 is a 0 value. Then, if you have the summation term as a positive value, which is $\max(0, 4)$, since the maximum value between these two is a 4, so the output will be 4. This is So, what are we trying to understand from this particular activation function? Is it not wherever it is a negative value, probably that particular feature is not too much contributing for the model prediction. Is it not ReLU is suppressing some part of the information? Only positive information

[07:21] is being sent. That's what we are understanding from this particular. But, there is one disadvantage with this. What is that a disadvantage? Is it not even when there is a negative value, ReLU is suppressing it? Is it not a dying neuron problem here? This particular we call it as a dying neuron. So, what is the meaning of dying a neuron? Suppose if this a neuron has a some summation value which is having a negative which is minus three. When you pass it to a relu activation, the output is zero. Which means that the neuron did not give us any information, which is not ideally correct. So, how is that we are going to resolve this particular problem? So, therefore, we have a second activation function which is your called

[08:04] leaky relu. So, what exactly is a leaky relu? Can you able to see the difference between the first and then the second graph? In the first one, all the negative we have a straight line there. But, that is not correct. We want even the negative information also to be processed to further. If not the complete information, some portion has to be processed for the next. Therefore, there is a small change in the formula.

[08:30] Instead of max zero, here we are multiplying the input with a very small constant which is 0.01. Now, you can ask the question, why 0.01 only? Why not any other value? We can have any other value, then it is called parametric relu. It is not leaky relu. For a leaky relu, we have this as a constant value which is 0.01. Let us take the same example and then see how it is having an impact. When it was minus three for relu, we got zero output. But, that's not right. So, how is that we are modifying it? Since the value is less than zero, whenever the value is less than zero, we are going to multiply that value with that small constant. So, when you multiply minus three with 0.01, we are getting answer as minus 0.03.

[09:24] Uh even if you don't want to pass the entire information, at least the small information is being passed to the next. That is what is the advantage of leaky relu. So, what is that leaky relu has also taken care? ReLU had a problem of dying neuron. Is it not the leaky ReLU has a resolved that particular problem? Can you able to correlate and then understand? Yes? Okay. Next. Uh here in a leaky ReLU, if you see, the negative part it has a very sharp curve. So, now what if you want to smoothen that? Can you able to observe?

[10:00] This is very sharp curve. Now, what if I want to have a smoother one? So, then we have something called as ELU. So, how is that? What is ELU? Whenever you have a value less than or equal to zero, here you have multiplied that value with 0.01. But here we are taking the help of exponential. So, let us take the example. Here you see, if the value is a minus one. So, since minus one is less than zero, we are using this particular formula. Where alpha is a constant value that we have to fix, which is an hyperparameter. So, we here it is one. That one is getting multiplied with e to the power of x here is minus one. So, you have substituted the value and we have obtained this.

[10:49] Now, go back and then similarly take an example of minus three and get the value. Then you will be able to see a clear difference between leaky ReLU and ELU. So, this is the third activation function that we will be using at an a hidden layer. Then we have because it is a modern deep learning where the advancement is too much. Therefore, there are many new activation functions which has come up for this modern deep learning models. That is GELU.

[11:22] Now, what exactly is GELU? This is the formula for the which is an All these are some of the variants of ReLU only. Now, this is mainly used whenever you are going for a modern deep learning. For example, we can go for transformers. So, we

will see that in the upcoming slide where all we'll be using each of these activation functions. So, the formula for geLU is this. Now, in the neuron, we are going to get the summation term. That summation term, when you substitute, when you pass that value into this particular equation, this is what is the impact. Now, can you able to see this?

[12:03] If you have a minus one, substitute a minus one wherever we have x , and then calculate, we are going to get this output. Similar way, all other values. So, these are and we have again one more development which is Silu , silu , and this is the activation. This is a formula which is used which is an extension from It is a taking the inspiration from sigmoid and then it this particular activation function. All these are different activation functions which are used at the hidden layer. Now, from the previous slide, we have seen the activation functions are something which can be used at a hidden layer as well as at the output layer. So, these are some of the activation functions that you can use at the hidden layer. If that is the

[12:51] case, what are the different activation functions we use at an output layer? Do we have choice here? Yes, answer is yes. Based on what kind of problem that we are solving, we have to choose a particular activation function. Suppose, if you are choosing an if you are performing an classification problem, more specifically if you are performing an binary classification, what is binary? It is a two. If it is a two, the answer is either yes or no, whether a particular person is a suffering from a disease or not. That's a classification problem. E- Here, the output is either S or no, one or zero. So, that is the reason why sigmoid is used. Now, you can ask how sigmoid is achieving this.

[13:36] So, once again, we have a summation term. Let us assume that summation value we are denoting it as X . Let us take an example where X is a minus two. When we take X as a minus two, just substitute the values of that into the equation. So, it is $1 / 1 + e$ to the power of minus X . Since here X is minus two, minus of minus is positive. So, is it not we got after substituting the values, we are getting the answer as 0.119.

[14:10] Now, you can ask from this, how are we classifying it as either one or zero? Go back to the sigmoid graph. Can you able to see the sigmoid graph has a cut at 0.5. Now, since the value what we have obtained is 0.1, which is less than 0.5, therefore this belongs to a class zero. I hope you are getting this, right? So, similar way, you can take another examples also. We can take what in the value is zero. If the value is zero, it is a either it can be towards zero class or it can be one class because we are getting the answer as 0.5. When it is 0.5, either it is towards one or it is towards zero based on how you have you know, documented it or how you have implemented it.

[15:01] Suppose, if the value is positive or X value here what we have considered is two. So, substitute the value $1 / 1 + e$ to the power of minus X , which is minus

two. And then the value what we are getting is 0.881. Definitely, this value is greater than 0.5. Therefore, this belongs to a class one. So, sigmoid is helping us in order to understand whether it belongs to zero class or one class. Therefore, whenever we are dealing with the binary classification problem, the activation function that we'll be using is sigmoid.

[15:39] So, this is one, but what if you ask the question, what if I have an output as minus one and one, not as zero and one? The two possibilities is if you have the output in terms of minus one and one, one option is come back substitute or replace all minus one to zero so that you can still use a sigmoid. But what if you don't want to change? You want output as minus one only. Then we cannot use a sigmoid. You need another activation function. That activation function is a tan h. And what is the formula for tan h? It is e^x minus e^{-x} divided by the same term but with positive sign. So, let's take some examples here.

[16:27] When you see minus two, tan h is giving us the output minus 0.964. When you give zero, it is zero. When it is a two, we are getting a positive value. Then here when the value is a greater than 0.5, you have rounded it off to one. When the value is less than 0.5, you set it as zero. Then in tan h, how any value which is a greater than zero belongs to plus class, any plus one, any value which is less than zero zero will be minus. So, here this is less than zero, therefore this is minus one class.

[17:05] Whereas when you take 0.964, the value is greater than zero, so this is a positive one. So, you're able to understand, right? Tan h is also used for binary classification, but if your output is -1 and 1. Now, in a real world, it is just not binary classification. What if you have a multi-class? Suppose we have a medical application where we want to categorize the person as a normal, then either benign or malignancy. So, if you have a multiple classes, or if you take an example as a male, female, and transgender. When you have a multi-class, not just two, multiple classes, then for sure sigmoid and TanH cannot be used. If that is the case, what is the next activation function that we have? It is the

[17:57] softmax. And what is the formula for softmax? Rather than directly looking into the mathematical one, let us take an example and then understand. Now, what exactly is this softmax? If you look into this example, here how many number of neurons we will place at an output layer will be based on number of classes. Suppose if the class is five, five different classes we have, then is it not we are going to have five neurons at an output layer? The answer is yes, we will have five. If we have a five neurons, in every of this we are going to get the summation part. You agree to this? Yes. Now, whatever is that summation, that is this value. In the single vector what you are seeing, 1.3, 5.1, on up to 1.1, this is all the

[18:48] summation term. Now, that is that has to be passed to an activation function. What activation function we are using? Softmax. How is that softmax works? What

is this numerator? It is e to the power of 1.3 whole divided by rest of it. It is e to the power of 1.3 + e to the power of 5.1 then e to the power of 2.2 e to the power of 0.7, and e to the power of 1.1. So, this is what is the calculation that we are going to perform. So, whatever is this calculation, which is 0.02.

[19:25] Uh so, I expect you to calculate this and then see that you will be getting the answer 0.02. Similar way, when you take the second one, there is a change only in the numerator. So, instead of e to the power of 1.3, we will have e to the power of 5.1. Uh so, e to the power of 5.1 whole divided by summation of all of those terms. You're going to get the answer 0.90. Similar way, for the next you will be calculating e to the power of 2.2. And then, you will calculate this and you're going to get this particular answer 0.05. So on and so forth. Now, you can ask uh how softmax is helping us when we are dealing with multi-class classification. We have understood with sigmoid and tan h, but softmax? What is

[20:16] softmax? Suppose if I have five classes, say for example, class one, class two, three, four, and five class. Take this example as a some animal data set. C1 belongs to cat, C2 belongs to dog, C3 belongs to horse, so on and so forth. Now, we have got one particular input sample. Now, that input, the model has a predicted it as uh you know, it belongs to the class C2. That's how we have to say, right? The probability that the given input belongs to if C2 is a dog, it is a dog class.

[20:51] Now, how softmax is helping us? Can you able to see this probabilities? Here, among all, the second class probability is very high. Therefore, whichever wherever the position the probability is high, the particular sample belongs to that class. And we know when we say probability, sum of that should be one. So, is this clear that the different activation functions that are used at an output layer? So, a quick summary, sigmoid is one which will be used at a binary classification where the output will be between zero and one. Then tan h is used again for a binary classification where the output is minus one and one. And the soft max we will use for a multi-class classification.

[21:38] With this, we have come to an end of an you know, activation functions, but we should know how these activation functions are used in a different models. In this particular course, what all are the topics that we are covering? We are first starting with the CNN in the week one. Then we go to autoencoders, variational autoencoders. Then we get into GANs. Then we are moving into diffusion models. Then we will be looking into sequence models, LLMs.

[22:11] The flow of the course is something like this. Now, can we use the same activation functions for all of these models? The answer is no. Whenever we are dealing with the Now, we know that activation functions can be used at a hidden layer as well as at the output layer, right? Whenever we are dealing with CNN, autoencoders, VAE, and GAN, then in the hidden layer, these are the possible activation functions

that we can use. We can go with the ReLU, leaky ReLU, parametric ReLU, or ELU. Now, whenever we are going with the transformers and LLM models, then at a hidden layer, we will be using GELU or the variant of GELU. So, what exactly is this? It is a Gaussian error linear unit.

[23:02] So, either we can use GELU or the variant of it when we are dealing with the transformers or the LLM model. But, when we are dealing with the diffusion model, it is either SiLU or it is this particular hidden layer activation function that we will be using. This is all with respect to hidden layer. What about at the output layer, what activation functions we'll be using? Whenever we are dealing with auto encoders, VAE decoder, and GAN generator, at an output layer, we will use either sigmoid or tanh. When we are dealing with GAN discriminator, the at an output layer, we'll be using sigmoid activation function. Then, whenever we are dealing with LLMs, where our aim is to predict what is our next word, then we'll be

[23:53] using an softmax as an activation function. So, what is this slide trying to tell you? Whatever we have learned in the next two slides, we are trying to connect where it is used in our particular course. Next, let us let us see the next fundamental concept, which is a loss function. Now, what are different loss functions that are there in a neural network? And then, which loss function is used for which particular model? When you look into this, you have an input data. This input data is passed to a neural network, and a neural network is going to give us some output. That is, it is going to predict the output. Then, what is that we already have with us? We have the ground truth. Then, what is the loss

[24:40] function? Loss function is it is some function which will calculate how far is our ground truth to the predicted value? Or whatever the model has a predicted, how far it is from the ground truth? So, loss function is one which is going to take the predicted and the ground truth and it will compute what is the loss that the model has made. So now if that is the case, what are different loss functions we have? Can we use the same loss functions for all the applications? Definitely no. So let us see some of the different loss functions that are there.

[25:16] So loss function is something which will measure how far the model prediction is from the true output or from the ground truth. Now for this what is that given to us? Given is we have an input data and then the label. So this is what is the given input to us. Then what is the role of a loss? The loss is it has to calculate the loss between the predicted value and then the ground truth. Now what is how is that you are going to get this a predicted value? How is that we get this? Is it not at every of this neuron we are going to get the we are going to calculate the summation? Then we pass it to an activation. Again we are going to get the summation. Is it not? It is some function where input is

[26:02] getting multiplied with the weight and then there is a bias at the term. Yes?

So this is how you will be calculating the loss. Now we should understand what are different loss functions. Here I'm trying to connect the conventional deep learning with modern deep learning. So if you take in deep learning, uh you know the model learns how to you know compare the ground truth with the predicted value. So it is going to predict and then we are going to compare it with the ground truth. But and then the loss we are going to get in some numerical value and we are going to say how far it is. But when you take the modern deep learning which is generative AI. So it is just not predicting the value. What

[26:49] are the different objectives of modern deep learning? The objective is it should not predict it has to generate a new data. It has to reconstruct the input, right? Or it has to generate a realistic new samples or it has to predict what is the next word. When I say deep learning is the next word the model should be able to predict it. So it is just not predicting from the given input. The whole requirement of modern deep learning where more specifically generative AI is getting a new data all together. In that case, the loss function cannot be as simple as our deep learning loss functions. Here the loss functions what we have is reconstruction and the classification loss and second is generative model loss. Let's look into

[27:39] each of this one by one. So under the reconstruction and the classification loss we first have something called as mean square error. So MSE. Now you can immediately uh think we have seen something this similarly in our deep learning course. In what way this is different? Here the reconstruction loss what we are seeing in your ML course or in the deep learning you'd have studied about this as 1 by n in the place of X you will be having the ground truth and the predicted value.

[28:14] You'd have have in the place of X you will be having the ground truth that is the label and then the predicted value. But here what is our aim? Our aim is to reconstruct the data. Suppose if I give the data as 2 4 and 6 as some three features and those three features values are 2 4 6. When I want the output, I want output something similar to 2 4 6 only which means I am regenerating I'm reconstructing this input. I'm generating a new data. So in that case, what is the ground truth for this? The original input. Look into the formula.

[28:53] This x_i what we have written is the original input. And then this $\hat{x}_i$ or $\hat{x}_i$ is the reconstructed input. Now, if I have the original input as 25, if the reconstructed one is 20, is it not I now need to see how far is the reconstructed one to the original? The difference is a five. So, how are we doing? This is the original x_i , this is the reconstructed one $\hat{x}_i$. We are taking the difference between these two.

[29:24] So, reconstruction loss is mean square error only, but we are not dealing with the original labels, but we are dealing with the original samples itself, original input itself. Yes. Now, this is with respect to if you are trying to reconstruct the continuous data. But what if there is a requirement of reconstructing the binary data? What if

you want in terms of zero and one? I don't want 25, 28. I don't want continuous value. I want just the answer as zero one. Then we cannot go for this mean square error. Then what is the error we have? We have a binary cross entropy. Once again, what you have already seen. But what you have seen, what you have learned already from your other courses or from the machine

[30:16] learning or from the deep learning course, from this there we had seen in the place of x_i we were dealing with the original labels, but here we are dealing with original input. So, once again, we'll be using binary cross entropy loss. Next, what if your requirement is to predict the next word? So, when you are dealing with the LLM's application, Uh you take all the uh you know, GPT models. There the aim is it has to predict the next word. In that case, what is the loss function that you will be doing? The predicting the next word is among the set of possible words, what is the most appropriate word, right? So, it is a kind of a classification only that you are doing. Therefore, you are using an

[31:03] cross entropy loss. In that cross entropy, suppose if you have uh encode If you have used one hot label encoding, then the loss function that you'll be using is categorical cross entropy. If it is a categorical cross entropy, this is the formula that you will be using. Where you know, and more specifically, it will be used in LLMs. Where the aim is to predict the next token. Similar way, instead of using one hot encoding, what if you are using label integer labels only? For example, we have deep learning. Let me give an example.

[31:44] Deep learning is your aim is you have to predict the next word. In the next word, you will have the possible words already. From those possible words, one of the most appropriate word will come here, right? Now, when you have among the set of you're picking one, is it not a classification? If it is a classification, these are set if you are able to if you are using it as one hot encoding, it is categorical cross. If you are using it as a label only, the first word is one, second word is two, third word is three, fourth one is four. Instead of using one hot, you are using as integer numbers only, then the loss function that we use is sparse categorical cross entropy. However, all this we'll be seeing in the hands-on

[32:34] session, but the it is in week one where you have to know all the fundamental concepts, you are learning this in a theory session. Once again, towards the end of this week in the hands-on, you'll be seeing all of these concepts. Yeah. So, the next is what are the different loss functions that are there at an generative models? So, the loss functions are we have a variational for at an for a model variational autoencoder, the loss function that we use is reconstruction loss plus KL divergence. We have something called as KL divergence. Now, what is KL divergence? You can able to see this. So far, our requirement was what is the input? What is the output that it has produced and you were finding out the

[33:23] difference between these two. So, that is your reconstruction loss. When it is the KL divergence, what is KL divergence? You are measuring how far one probability distribution is different from other probability distribution. For example, when I take there is one probability distribution like this, there is one more probability distribution which is like this. So, these are two are two probabilities. Now, if I want to measure how far these two probabilities, then we will be using a KL divergence. Now, if you want to understand this further, just imagine in a two-dimensional space, you have a two points P and Q. If you want to measure the distance between these two, we can use Euclidean distance, right? Why?

[34:12] Because P will have points coordinates X and Y and the other Q also will have X and Y coordinates. So, with this, you can able to measure how far P and Q with Euclidean distance. But now, here we don't have a points. Here we have a probability. We have a distributions here. This is a Gaussian distribution. This is an Bernoulli distribution. So when you have two different distributions, how far is one distribution to another? We cannot use Euclidean. Why? Because that requires a points. But when we are dealing with a distribution, the loss function that we will be using is KL divergence, which we will once again see in detail during our week three session.

[34:58] Yeah. So the next uh loss function what we have is again. So this is a generative model, GAN, and the loss function that we are going to see here is how the the loss function. Once again, during our week four, we are going to see this mathematically. But for this particular session, you try to understand that generator try to generate the data, and the aim of generator is discriminator is the one which is going to discriminate between which is real, which is a fake. So what should be the role of a generator?

[35:32] Generator should generate the images as realistic as possible so the discriminator gets confused whether it's real or fake. So this is what is a GAN, which will be seeing, and the loss function we have a loss function which will be you know, which is used in GAN. And the last is a diffusion model, which is once again in a generative model. And in the diffusion, we have something called as noise prediction loss. So these are different loss functions that we have. So with this, we have come to an end of this particular session. And in this session, we have mainly focused on understanding what are different activation functions we have, and what are different loss functions. So with this, we will come to we have come to an

[36:20] end of this session. In the upcoming session, we will be seeing what are the different optimizers that are there in the neural network. Thank you.